

CSS Grid Items Alignment Properties

Complete Guide: Easy English Explanation

CSS Grid Mastery

May 10, 2026

Contents

1	Introduction	2
1.1	What is CSS Grid Item Alignment?	2
1.2	Why is Alignment Important?	2
2	Core Grid Alignment Properties	3
2.1	justify-items Property	3
2.1.1	What Does It Do?	3
2.1.2	Available Values	3
2.1.3	Basic Example	3
2.2	align-items Property	3
2.2.1	What Does It Do?	3
2.2.2	Available Values	4
2.2.3	Basic Example	4
2.3	justify-content Property	4
2.3.1	What Does It Do?	4
2.3.2	Available Values	4
2.3.3	Example	4
2.4	align-content Property	5
2.4.1	What Does It Do?	5
2.4.2	Available Values	5
2.4.3	Example	5
3	Individual Item Alignment	6
3.1	justify-self Property	6
3.1.1	What Does It Do?	6
3.1.2	Values	6
3.1.3	Example	6
3.2	align-self Property	6
3.2.1	What Does It Do?	6
3.2.2	Values	7
3.2.3	Example	7
3.3	The place-self Shorthand	7
4	Real-World Examples	8
4.1	Example 1: Card Grid (Equal Spacing)	8
4.2	Example 2: Dashboard with Centered Content	8
4.3	Example 3: Mixed Alignment	9

5 Quick Reference Table	10
6 Common Mistakes & Solutions	11
6.1 Mistake 1: Confusing Container vs Item Properties	11
6.2 Mistake 2: Forgetting About Default stretch	11
6.3 Mistake 3: Mixing Grid and Flexbox Thinking	12
7 Browser Support	13
8 Summary: Key Takeaways	14
8.1 Remember These Points	14
8.2 Quick Decision Guide	14
9 Conclusion	15

Chapter 1

Introduction

1.1 What is CSS Grid Item Alignment?

CSS Grid gives you powerful tools to position and align items inside a grid container. When you create a grid, items can be aligned in two directions:

- **Horizontally (left to right)** — using properties on the row axis
- **Vertically (top to bottom)** — using properties on the column axis

Think of alignment like placing objects on a chessboard — you control where each piece sits in its square.

1.2 Why is Alignment Important?

- Creates professional, organized layouts
- Saves time — no need for manual positioning
- Makes responsive design easier
- Improves visual hierarchy and readability
- Reduces spacing calculations

Chapter 2

Core Grid Alignment Properties

2.1 justify-items Property

2.1.1 What Does It Do?

The `justify-items` property aligns grid items **horizontally** within their grid cells. It's applied to the grid **container**.

2.1.2 Available Values

Value	Explanation
<code>start</code>	Aligns items to the left edge of the cell
<code>end</code>	Aligns items to the right edge of the cell
<code>center</code>	Centers items horizontally
<code>stretch</code>	Stretches items to fill the cell (default)

2.1.3 Basic Example

```
.grid-container {  
  display: grid;  
  grid-template-columns: 100px 100px 100px;  
  justify-items: center;  
  gap: 10px;  
}  
  
.item {  
  background-color: #3498db;  
  padding: 20px;  
}
```

Result: All grid items are centered horizontally in their cells.

2.2 align-items Property

2.2.1 What Does It Do?

The `align-items` property aligns grid items **vertically** within their grid cells. It's applied to the grid **container**.

2.2.2 Available Values

Value	Explanation
start	Aligns items to the top edge of the cell
end	Aligns items to the bottom edge of the cell
center	Centers items vertically
stretch	Stretches items to fill the cell (default)

2.2.3 Basic Example

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px;
  grid-template-rows: 150px 150px;
  align-items: center;
  gap: 10px;
}

.item {
  background-color: #2ecc71;
  padding: 20px;
}
```

Result: All grid items are centered vertically in their cells.

2.3 justify-content Property

2.3.1 What Does It Do?

The **justify-content** property aligns the **entire grid** horizontally within the container. It controls space around all columns together.

2.3.2 Available Values

Value	Explanation
start	Aligns grid to the left
end	Aligns grid to the right
center	Centers the entire grid
space-between	Equal space between columns
space-around	Equal space around columns
space-evenly	Equal space everywhere
stretch	Stretches columns to fill container (default)

2.3.3 Example

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px 100px;
  justify-content: space-between;
  gap: 10px;
}
```

```
width: 400px;
border: 1px solid #ccc;
}

.item {
  background-color: #e74c3c;
  padding: 20px;
}
```

Result: The grid columns are spaced evenly with equal gaps between them.

2.4 align-content Property

2.4.1 What Does It Do?

The `align-content` property aligns the **entire grid** vertically within the container. It controls space around all rows together.

2.4.2 Available Values

Value	Explanation
<code>start</code>	Aligns grid to the top
<code>end</code>	Aligns grid to the bottom
<code>center</code>	Centers the entire grid
<code>space-between</code>	Equal space between rows
<code>space-around</code>	Equal space around rows
<code>space-evenly</code>	Equal space everywhere
<code>stretch</code>	Stretches rows to fill container (default)

2.4.3 Example

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px;
  grid-template-rows: 80px 80px;
  align-content: center;
  gap: 10px;
  height: 300px;
  border: 1px solid #ccc;
}

.item {
  background-color: #f39c12;
  padding: 20px;
}
```

Result: The entire grid is centered vertically in its container.

Chapter 3

Individual Item Alignment

3.1 justify-self Property

3.1.1 What Does It Do?

The `justify-self` property aligns a **single item** horizontally. It's applied to the item itself, overriding `justify-items`.

3.1.2 Values

`start`, `end`, `center`, `stretch`

3.1.3 Example

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px 100px;
  gap: 10px;
}

.item {
  background-color: #9b59b6;
  padding: 20px;
}

.item-special {
  justify-self: center; /* Only this item is centered */
  background-color: #e67e22;
}
```

Result: One specific item is centered while others follow default alignment.

3.2 align-self Property

3.2.1 What Does It Do?

The `align-self` property aligns a **single item** vertically. It's applied to the item itself, overriding `align-items`.

3.2.2 Values

start, end, center, stretch

3.2.3 Example

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px;
  grid-template-rows: 150px 150px;
  gap: 10px;
}

.item {
  background-color: #16a085;
  padding: 20px;
}

.item-special {
  align-self: center; /* Only this item is vertically centered */
  background-color: #1abc9c;
}
```

Result: One specific item is vertically centered.

3.3 The place-self Shorthand

Combine justify-self and align-self in one property:

```
.item {
  place-self: center center; /* horizontal, vertical */
}

/* Same as: */
.item {
  justify-self: center;
  align-self: center;
}
```

Chapter 4

Real-World Examples

4.1 Example 1: Card Grid (Equal Spacing)

```
.card-grid {  
  display: grid;  
  grid-template-columns: 150px 150px 150px;  
  justify-items: center;  
  align-items: center;  
  gap: 15px;  
  padding: 20px;  
}  
  
.card {  
  background: white;  
  border: 1px solid #ddd;  
  border-radius: 8px;  
  padding: 15px;  
  text-align: center;  
}
```

Use case: Product listings, portfolio items, or team member cards.

4.2 Example 2: Dashboard with Centered Content

```
.dashboard {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;  
  grid-template-rows: 200px 200px;  
  justify-content: space-around;  
  align-content: space-around;  
  gap: 20px;  
  min-height: 500px;  
  background: #f5f5f5;  
}  
  
.widget {  
  background: white;
```

```
border-radius: 10px;
padding: 20px;
box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
```

Use case: Admin dashboards with distributed widgets.

4.3 Example 3: Mixed Alignment

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px 100px;
  grid-template-rows: 100px 100px;
  gap: 10px;
}

.item {
  background: #3498db;
  padding: 20px;
}

.item-left {
  justify-self: start; /* Align left only */
}

.item-center {
  place-self: center; /* Center both ways */
}

.item-right {
  justify-self: end; /* Align right only */
}
```

Use case: Complex layouts with varied alignment needs.

Chapter 5

Quick Reference Table

Property	Applied To	Direction	What It Does
<code>justify-items</code>	Container	Horizontal	Aligns ALL items left/right/center
<code>align-items</code>	Container	Vertical	Aligns ALL items top/middle/bottom
<code>justify-content</code>	Container	Horizontal	Aligns entire GRID left/right/center
<code>align-content</code>	Container	Vertical	Aligns entire GRID top/middle/bottom
<code>justify-self</code>	Item	Horizontal	Aligns ONE item left/right/center
<code>align-self</code>	Item	Vertical	Aligns ONE item top/middle/bottom
<code>place-items</code>	Container	Both	Shorthand for <code>justify-items</code> + <code>align-items</code>
<code>place-content</code>	Container	Both	Shorthand for <code>justify-content</code> + <code>align-content</code>
<code>place-self</code>	Item	Both	Shorthand for <code>justify-self</code> + <code>align-self</code>

Chapter 6

Common Mistakes & Solutions

6.1 Mistake 1: Confusing Container vs Item Properties

Wrong:

```
.item {  
    justify-content: center; /* Wrong! This is for container */  
}
```

Correct:

```
.grid-container {  
    justify-content: center; /* Applied to container */  
}  
  
/* OR for one item: */  
.item {  
    justify-self: center; /* Applied to item */  
}
```

6.2 Mistake 2: Forgetting About Default stretch

Problem: Items fill the entire cell by default, making alignment invisible.

Solution:

```
.grid-container {  
    grid-template-columns: 200px 200px;  
    grid-template-rows: 150px 150px;  
}  
  
.item {  
    width: fit-content; /* Prevents stretching */  
    justify-self: center;  
    align-self: center;  
}
```

6.3 Mistake 3: Mixing Grid and Flexbox Thinking

Remember:

- **Grid:** Two-dimensional (rows AND columns)
- **Flexbox:** One-dimensional (row OR column)

Both use similar property names but grid offers more control.

Chapter 7

Browser Support

All modern browsers support CSS Grid alignment properties:

Property	Chrome	Firefox	Safari	Edge
<code>justify-items</code>	57+	55+	10.1+	16+
<code>align-items</code>	57+	55+	10.1+	16+
<code>justify-content</code>	57+	55+	10.1+	16+
<code>align-content</code>	57+	55+	10.1+	16+
<code>justify-self</code>	57+	55+	10.1+	16+
<code>align-self</code>	57+	55+	10.1+	16+

Chapter 8

Summary: Key Takeaways

8.1 Remember These Points

- **Justify** = Horizontal alignment (left/right)
- **Align** = Vertical alignment (top/bottom)
- **-items** = Aligns individual items in cells
- **-content** = Aligns entire grid in container
- **-self** = Overrides container settings for one item
- Container properties apply to ALL items (except overridden by -self)
- Item properties override container properties

8.2 Quick Decision Guide

1. Want to center ONE item? → Use `place-self: center`
2. Want to center ALL items? → Use `place-items: center`
3. Want to center the ENTIRE GRID? → Use `place-content: center`
4. Want specific spacing between columns? → Use `justify-content`
5. Want specific spacing between rows? → Use `align-content`

Chapter 9

Conclusion

CSS Grid alignment properties give you precise control over layout without complicated math or hacks. By understanding the difference between items, content, and the direction (justify vs align), you can create professional, responsive layouts easily.

Start simple with `place-items: center` to center everything, then gradually use more advanced properties as your layouts become more complex.

Happy grid building!